Database Engineering: Keys, Relationships, Indexes, and Transactions

A relational database stores related facts in tables made of rows and columns. A primary key uniquely identifies each row in its table and must not be null. A foreign key references a candidate key, commonly a primary key, in another table. The database can enforce referential integrity so that a reference cannot point to a row that does not exist. A foreign key does not have to use the same column name as the referenced key, but its values must be compatible.

A one-to-many relationship means one row on the parent side can be related to many rows on the child side, while each child row refers to one parent. The foreign key is normally stored on the many side. For example, one conversation can have many messages, and each message can store the conversation's identifier. A many-to-many relationship allows rows on either side to relate to multiple rows on the other side. It is represented with a junction table containing foreign keys to both participating tables; a uniqueness constraint on the pair can prevent duplicate associations.

An index is an auxiliary data structure that can help a database find rows without scanning every row in a table. An index can improve reads and joins when its columns match useful filters or ordering, but it consumes storage and must be maintained when rows are inserted, updated, or deleted. Indexes do not guarantee a faster query: the optimizer chooses a plan based on the query and available statistics. Frequently filtered ownership identifiers are common index candidates in a multi-user application.

A database transaction groups operations into a unit that is committed or rolled back. ACID describes four transaction properties. Atomicity means all operations in the transaction take effect or none do. Consistency means a committed transaction preserves declared rules and constraints. Isolation controls how concurrent transactions observe one another's work. Durability means committed changes survive a failure according to the database's guarantees. ACID does not automatically include work in an external service such as a vector database; applications need retries, idempotency, or compensating cleanup for cross-system workflows.

SQL joins combine rows from tables using a related condition. An INNER JOIN returns rows that satisfy the join condition on both sides. A LEFT JOIN keeps every row from the left input and adds matching right-side values, using nulls when no right row matches. A join can combine messages with their conversation or a junction table with its documents. The join condition should use the intended key relationship; omitting it can create a Cartesian product with many unintended row combinations.

Schema constraints and application authorization solve different problems. A foreign key can ensure that an association references existing rows, while an application query must still filter by the authenticated user's ownership. For example, a junction table may carry a user identifier and use composite foreign keys to ensure its conversation and document belong to that same user. The API should also validate ownership before creating the association, and retrieval should apply the authenticated user filter every time.